

Private Intelligence

How XYRA AI Was Built to End the Era of Manual Engineering

By Prashanth Thipparthi, Founder - XYRA AI

XYRA AI - Private AI for EPC Engineering

Foreword: A Problem Hidden in Plain Sight

There is a scene that plays out in engineering offices all over the world, every single day.

A senior instrumentation engineer - ten, fifteen, twenty years of experience - opens a P&ID on one monitor and an empty Excel spreadsheet on the other. The drawing is dense: hundreds of instrument bubbles, tag numbers, line references, equipment callouts, and annotation notes crowded across a single sheet. Their job, for the next four hours, is to read every bubble, decode every tag, cross-reference every line number, and type the results, one row at a time, into a file that already has forty-seven previous entries from yesterday.

When they finish, that file will be emailed to the piping team, who will open it, copy the line numbers they need, paste them into a different spreadsheet formatted in a completely different way, and send that to procurement, who will re-enter the relevant data into their own system. The controls team will receive yet another version - possibly the wrong revision - and spend the first hour of their next meeting establishing which file everyone is actually working from.

This is not an exceptional project. This is every project.

And nobody built a tool to fix it. Not because the problem was invisible - every engineer in the room knows exactly how broken this workflow is - but because building the right fix is genuinely hard. It requires deep knowledge of ISA-5.1 instrumentation standards, P&ID drawing conventions, the EPC data model, and the immovable constraint that client drawings cannot leave the client's network. It requires not just AI, but the right kind of AI: context-aware, engineering-literate, and deployable inside a corporate firewall.

XYRA AI was built to be that fix.

This book is its story.

Chapter One: The Industry That Built the Modern World

Before understanding XYRA AI, you need to understand the world it was built for.

EPC - Engineering, Procurement, and Construction - is the industry that builds the infrastructure the modern world runs on. Refineries. Chemical processing plants. Offshore oil and gas platforms. LNG terminals. Power generation facilities. Desalination plants. Pharmaceutical manufacturing sites. Water treatment works.

These are not simple structures. A mid-scale oil refinery might contain fifty thousand individual instruments: pressure transmitters, flow controllers, temperature sensors, valve position indicators, safety shutdown switches, analytical devices. Every one of them needs to be identified, specified, sourced, installed, calibrated, tested, commissioned, and documented. Every instrument belongs to a loop. Every loop connects to a control system. Every control system interfaces with a safety layer. Every safety layer must be proven compliant before a single barrel of product flows.

The P&ID - the Piping and Instrumentation Diagram - is the master drawing that captures all of this. It is the engineering source of truth. Everything else - procurement lists, control system databases, instrument indexes, cause and effect matrices, safety system designs - ultimately derives from what is drawn on the P&ID.

A large EPC project might have eight hundred P&IDs. Each drawing can carry between fifty and three hundred individual instrument tags. The instrumentation team's first major deliverable - the Instrument Index - must capture every tag from every drawing: its type, its loop, its service description, its P&ID reference, its signal type, its IO assignment, its engineering unit, its design pressure and temperature, and its position in the control philosophy.

Do the arithmetic. Eight hundred drawings, one hundred and fifty instruments per drawing average: one hundred and twenty thousand individual instrument records, each requiring multiple data points, all of which must be accurate, traceable, and agreed across six or seven different engineering disciplines.

And, until XYRA AI, every single one of those records was populated by a human, reading a drawing, typing into a cell.

The EPC industry built refineries, platforms, and plants that will stand for fifty years. It has not yet built a better workflow for the people who design them.

Chapter Two: The Hidden Cost Nobody Talks About

There is a number that circulates quietly among EPC project directors and engineering managers: thirty to fifty percent.

That is the share of a project's engineering budget that is consumed by what can only be described as clerical work. Data extraction. Data re-entry. Data reformatting. Data verification. Data transfer. Not engineering decisions - clerical overhead. On a two-hundred-million-dollar EPC project, that means sixty to one hundred million dollars of engineering budget spent on tasks that add no engineering value whatsoever.

The source of this waste is not laziness or poor management. It is structural. It is baked into the way engineering information moves through an EPC project.

Consider the lifecycle of a single instrument tag from the moment a P&ID is issued.

An instrumentation engineer manually extracts the tag from the drawing: they read the bubble, decode the ISA-5.1 identifier, identify the loop, note the connected line number, infer the service from context, and enter the record into the Instrument Index spreadsheet. This takes, on average, two to four minutes per tag on a clean vector PDF. On a scanned drawing, it can take longer. A single P&ID with one hundred and fifty tags occupies a disciplined engineer for between five and ten hours.

The same tag is then entered into the client's PDMS or SPI database - a different system, a different format, a different team. Then into the procurement system for instrument datasheets and vendor selection. Then into the control system design tool for IO assignment and loop drawing generation. Then into the safety system documentation for SIS classification. Then into the commissioning database for field verification.

Each entry is a fresh opportunity for human error. A transposed digit. A misread qualifier. An instrument type decoded incorrectly. A line number associated with the wrong loop. These errors - small individually, catastrophic in aggregate - multiply across thousands of tags and hundreds of drawings until the Instrument Index, the control system database, and the procurement list are quietly inconsistent in ways that only surface during commissioning, when fixing them costs ten times what it would have cost at extraction.

The industry has accepted this as the cost of doing business. It should not have to.

Chapter Three: Data That Gets Lost Between Disciplines

If the extraction problem is the first crisis of EPC data management, the transfer problem is the second - and in some ways the more damaging one.

Engineering data on an EPC project does not flow. It fragments.

The instrumentation team completes a revision of the Instrument Index. They email it to the piping team. The piping team needs the line list - which is embedded in a different tab of the same file, formatted in a way that does not match the piping team's own templates. So the piping engineer copies the relevant columns, reformats them, and pastes them into a new file, which is then emailed to the mechanical team.

Two weeks later, the instrumentation team issues a revised Instrument Index. They have added twenty-three instruments from a new P&ID revision, corrected six tag numbers, and updated fourteen service descriptions. The email goes out. The piping team, who is now three revisions behind on their own deliverables, does not immediately update their working file. The mechanical team is working from a version they received before the revision.

By the time this is caught - usually in a review meeting - the discrepancy has propagated into four different documents owned by three different teams. Resolving it requires a coordination meeting, a document register update, and two or three hours of re-verification work.

This scenario is not rare. On large projects, some version of this plays out every week.

The root cause is not human error. It is the absence of a shared engineering record. Every discipline maintains its own version of the truth, synchronized manually through email, with no version control, no audit trail, and no automated notification when the source data changes.

The same instrument tag - FIC-1762P-12, for example, a flow indicating controller on a chemical injection line - might exist in twelve different files, in twelve slightly different forms, maintained by twelve different engineers, none of whom can be certain their version is current.

XYRA AI was designed with this problem as its second-order priority. The first is extraction accuracy. The second is shared truth - a single, structured engineering record that every discipline reads from and writes to, updated automatically as drawings are processed and revised.

Chapter Four: Why Generic AI Failed Engineering

In 2023 and 2024, as large language models became mainstream, engineering firms - like every other industry - began exploring whether general-purpose AI could help with the data problem.

The experiments were largely disappointing.

The failure modes fell into three clear categories, and understanding them is essential to understanding why XYRA AI had to be built as a purpose-built engineering tool rather than a wrapper around a general model.

The knowledge gap. ISA-5.1 - the Instrument Society of America standard for instrumentation identification - defines a precise encoding system for instrument tags. The first letter identifies the measured or initiating variable. The subsequent letters define the instrument's function. Qualifiers like HH and LL indicate alarm positions. Loop numbers tie instruments to control loops. Modifier letters indicate specific characteristics.

A flow indicating controller is an FIC. A pressure indicating transmitter is a PIT. A hand switch is an HS. A solenoid valve is a ZV. The decode is systematic but deeply domain-specific, with ambiguities that require engineering context to resolve. A general-purpose LLM trained on internet text has exposure to this system but no deep mastery of it. It will hallucinate plausible-sounding but incorrect classifications. In engineering, a hallucinated IO type or a misclassified instrument function is not an inconvenience - it is a commissioning error or, in a safety-critical system, a potential incident.

The drawing problem. P&IDs are complex engineering drawings. They are dense, layered, drawn to specific standards, and contain information that is meaningful only in spatial and relational context. An instrument bubble's position relative to a pipe line tells you which line it belongs to. The geometry of the connection tells you whether it is a primary or redundant measurement. The proximity to a valve tells you whether it is a position indicator or a control element.

General-purpose vision models can extract text from images. They cannot reliably interpret engineering drawing semantics. They see pixels. They do not understand that the circle with FCV-1762P-12 is not just a label - it is a flow control valve on a specific pipe, in a specific loop, performing a specific function in a process context.

The confidentiality wall. Most EPC clients - particularly those in oil and gas, chemical processing, and defence-adjacent industries - operate under strict data governance requirements. P&IDs are proprietary. They contain process design, safety system topology, and commercial data. Uploading them to a public AI service is not simply inadvisable - on many projects, it is a contractual violation.

This means that any AI tool that processes P&IDs must run inside the client's network, on the client's infrastructure, with no data leaving the controlled environment. General-purpose cloud AI is categorically excluded.

These three constraints - the knowledge gap, the drawing problem, and the confidentiality wall - define the design requirements for any AI that could actually work in EPC. XYRA AI was designed to satisfy all three from the first line of code.

Chapter Five: The Origin - Built by Someone Who Lived the Problem

XYRA AI was not founded in a university lab or a venture capital incubator. It was founded by a practitioner who spent years inside EPC engineering workflows and reached a point where the inefficiency became impossible to accept.

The insight at the heart of XYRA AI is simple but took years of domain experience to fully articulate: the tools that engineering teams use to process drawings were not designed for the drawings themselves. They were designed for general document management, adapted for engineering with varying degrees of success, and extended over decades through workarounds and manual conventions.

No one had ever built a tool that started with the engineering drawing as the primary artifact and designed everything - the extraction logic, the classification model, the output format, the deployment architecture - around the specific, well-understood requirements of EPC instrumentation and piping work.

The founding question was not "how do we apply AI to engineering?" It was "if you knew everything about how EPC engineering actually works - the ISA standards, the P&ID drawing conventions, the EPC data model, the instrument lifecycle from design to commissioning - what tool would you build?"

The answer was XYRA Studio.

The decision to build it as an on-premise, locally-deployed system was not a technical compromise - it was the correct architectural choice for this domain. Client data stays with the client. Models run on local hardware. The system works inside a corporate network with no external dependencies. This is not a limitation; it is the product's most important feature.

The decision to use fine-tuned local LLMs - built on Qwen2.5 7B, customised with engineering-specific knowledge through Modelfiles - rather than general-purpose cloud models was similarly intentional. A local model that deeply understands ISA-5.1, EPC drawing conventions, and instrument classification logic will consistently outperform a general cloud model on engineering tasks, while providing the privacy guarantees that the industry requires.

XYRA AI was built because someone decided that the engineering world deserved tools built specifically for it - not adapted from tools built for other purposes, but designed from scratch with domain knowledge as the foundation.

Interlude: The Company Behind the Platform

XYRA AI is the company built around that conviction.

It is not a generic AI software company looking for an industry problem to attach itself to. It is an engineering technology company focused on EPC workflows: the practical, document-heavy, data-fragmented world of instrumentation, piping, process, project controls, procurement, commissioning, and handover.

The public face of that company is xyra-ai.com.

The website exists for a simple reason: to show the engineering world what XYRA is building, how the tools connect, and how a client can engage. It presents the company in three layers.

The first layer is XYRA Studio: the flagship private AI platform for EPC engineering teams. Studio is the licensed, client-hosted product. It is installed inside the client environment, runs on client-controlled infrastructure, and is designed for projects where drawings, instrument data, and engineering records cannot be uploaded to public cloud services.

The second layer is the focused product ecosystem around Studio. InstruMap handles P&ID instrument extraction. PrecisionPDF supports drawing review and PDF workflows. DataPump helps move structured data into SmartPlant Instrumentation databases. DataDiff compares revisions of engineering Excel files. FlowSizing supports instrument and process sizing calculations. UnitMaster handles daily engineering unit conversions. These tools are useful independently, but they also communicate the broader direction of XYRA AI: engineering work should be structured, traceable, automated where possible, and always reviewable by engineers.

The third layer is engineering and technology services. Not every client problem is solved by a product out of the box. Some clients need SPI migration support, SPPID data extraction, AVEVA or Hexagon workflow automation, custom database utilities, or project-specific AI tools. Others need deeper extensions inside the engineering applications they already use: custom DLLs for Hexagon and AVEVA environments, controlled plug-ins, project utilities, validation tools, import/export automations, and workflow-specific add-ons that fit existing client standards.

XYRA AI also supports the quieter but important engineering software work that surrounds these systems: project setup for EPC companies on Hexagon and AVEVA tools, standards implementation, database configuration, schema mapping, template and report setup, DLL obfuscation and IP protection services, deployment packaging, data migration scripts, bulk update utilities, Excel-to-database workflows, engineering register cleanup, revision comparison, QA dashboards, and handover data preparation. These services sit close to the real project data, where small improvements in automation can save hundreds of engineering hours.

XYRA AI works in that space as a partner, building controlled systems around real project constraints rather than forcing every client into the same template.

This combination is intentional.

XYRA AI is a product company because the EPC industry needs durable tools, not one-off scripts that disappear after a project. It is also a services company because every EPC organisation has its own standards, data models, naming conventions, templates, security rules, and delivery practices. A serious engineering AI company must understand both: the repeatable product layer and the project-specific implementation layer.

The company is built on a practical belief: AI in engineering must earn trust before it earns scale.

That means outputs must be traceable. Every extracted tag should carry evidence. Every uncertain record should be flagged. Every review action should be visible. Every client deployment should respect the confidentiality wall around project data. Every automation should leave the engineer in control.

This is why XYRA AI does not position AI as a replacement for engineering judgment. The company exists to remove the clerical load around that judgment: extracting, counting, comparing, formatting, mapping, validating, and preparing the first structured draft of the work.

The engineer still decides. XYRA AI makes sure the engineer starts from better evidence.

That is the company behind XYRA Studio: a focused EPC engineering intelligence company, building private AI systems for the parts of engineering work that general software never understood.

Chapter Six: XYRA Studio - One Platform, Every Tool

XYRA Studio is the product that XYRA AI delivers to EPC engineering teams. It is a browser-based platform deployed inside the client's network - a single workspace where instrumentation engineers, piping engineers, and project managers access every drawing intelligence tool without re-uploading files or switching between disconnected systems.

InstruMap: Instrumentation Intelligence

InstruMap is the centrepiece capability. It processes P&ID PDFs - both vector-format (searchable) and scanned (OCR) - and extracts every instrument tag it finds.

The extraction is not simply optical character recognition. InstruMap applies ISA-5.1 classification logic to every detected tag, decoding the first letter and subsequent letters, resolving qualifier prefixes, identifying the instrument type, and flagging uncertain classifications for engineer review. It identifies pipe line numbers from the drawing geometry and maps each instrument to its connected line, inferring service descriptions from upstream and downstream context, nearby line identifiers, and project legend data.

The outputs are engineering deliverables, not data dumps:

- **Instrument Index:** Every detected tag with loop number, instrument type, service description, ISA classification, IO type, signal type, P&ID reference, line association, and QA flags for uncertain records.
- **IO List:** The same data organised by IO type - AI, AO, DI, DO - structured for controls and automation engineering review.
- **Verification Log:** The full extraction trail for every instrument, including raw extracted text, confidence score, suppression reason if filtered, and reviewer notes.
- **Line List:** All pipe line numbers detected in the drawing, with associated instrument records and engineering attributes.

These outputs are delivered as a ZIP package containing review-ready Excel files. An engineer can open the Instrument Index immediately after a batch run and begin reviewing - filtering by confidence, sorting by loop, checking flagged records - without any reformatting or preparation.

Piping MTO: Computer-Vision Component Detection

The Piping MTO tool solves a different but equally costly problem: the manual counting of piping components from P&IDs for material take-off purposes.

On a traditional project, a piping engineer identifies each valve type, fitting, specialty item, and equipment nozzle connection on every drawing by eye, tallying counts in a spreadsheet. On a five-hundred-drawing package, this is weeks of work with significant error potential.

XYRA Studio's MTO tool replaces this process with computer-vision detection. The engineer selects a component on any drawing - draws a box around a ball valve, for example - and XYRA saves it to the component library. From that single selection, XYRA detects every visually matching instance across all drawings, all pages, in all orientations (0°, 90°, 180°, 270°), extracting size annotations near each detection and compiling the results into an EPC-style Excel package.

The output includes a Piping Material Take-Off sheet, a Detection Register with page-level evidence for every match, QA Checks with automated threshold validation, and a run metadata file. All results are reviewable in the browser before export - engineers can remove false positives and add notes before the final package is generated.

PrecisionPDF: Drawing Review Workspace

PrecisionPDF is the drawing review and markup tool, built directly into the same workspace. Engineers can open any uploaded drawing, navigate via thumbnails and a minimap, search the text layer for tag numbers or line references, apply annotations and markup, and save the annotated PDF - all without leaving XYRA Studio or opening a separate viewer application.

System Health: Compute Fabric Monitor

The System Health dashboard gives clients real-time visibility into the state of their XYRA deployment. It shows the status of every service - API core, state bus (Redis), worker engine, LLM runtime - and the state of every custom XYRA model loaded in Ollama. The dashboard communicates clearly: 8/8 services live, 4 AI engines active, 18.7 GB of models loaded, 100% health.

It is not just a status page. It is a demonstration that the client's local compute fabric is running, that the AI is present and loaded, and that any extraction or detection job the engineer submits will be processed by a capable local system - not sent to a cloud service, not queued behind external infrastructure, not dependent on an internet connection.

Chapter Seven: The Technology Underneath

XYRA Studio's capabilities rest on a technology architecture designed specifically for the EPC context: local, private, efficient, and recoverable from failure without losing the engineer's work.

Local LLM Infrastructure

The AI engine at XYRA's core is Ollama - an open-source local LLM runtime that manages model serving on the client's hardware. Ollama runs on the same server as the rest of the XYRA Stack, with no external network calls.

The base model is Qwen2.5 7B - a capable, efficient open-source language model. XYRA AI fine-tunes this model through Modelfiles that encode engineering-specific knowledge: ISA-5.1 first-letter decode tables, subsequent-letter function identifiers, qualifier logic for HH, LL, H, L alarm designations, EPC naming conventions, drawing context interpretation strategies, and examples drawn from real engineering scenarios.

This produces four custom models:

- xyra-pid-engineer: Instrument tag understanding, ISA-5.1 classification, noise rejection, confidence scoring.
- xyra-line-mapper: Instrument-to-line mapping decisions based on geometric proximity and drawing context.
- xyra-project-context: Project legend, title block, and scope extraction to guide the instrument identification process.
- xyra-mto-reviewer: MTO result review, QA analysis, and reviewer notes for detected component sets.

Each model is purpose-built for its task. Each has a fallback - if the custom model is unavailable, the system reverts to base Qwen2.5 7B, maintaining function at reduced accuracy rather than failing completely.

The Deployment Architecture

XYRA Studio runs as a Docker Compose application on a single server - a Windows Server or Linux machine inside the client's network. Five services run in Docker containers: the React frontend served by nginx (the only publicly exposed surface), the FastAPI backend, the RQ worker for background job processing, Redis as the job queue and state broker, and Ollama.

Only port 80 is exposed to the client network. Every other service - the backend API, the worker, Redis, Ollama - runs on Docker internal networks, invisible to the external network. An engineer's browser makes HTTP requests to nginx on port 80. nginx proxies API calls to the backend. The backend dispatches jobs to the worker through Redis. The worker calls Ollama for LLM inference. No request ever reaches the internet.

This architecture satisfies the most stringent client security requirements. P&IDs, extracted data, and all outputs remain on the client-controlled server. The system functions completely offline. A client with no internet connection can run a full extraction batch on five hundred drawings using four purpose-built AI models without a single packet leaving their facility.

Resilience and Recovery

A core design principle of XYRA Studio is that AI failures must not block engineering results. If the LLM inference fails for any reason - model timeout, memory pressure, service restart - the deterministic extraction pipeline continues and returns whatever results it has, with clear indication of what the AI contributed and what fell back. The engineer receives a usable output, even if some confidence scores or service inferences are missing.

Long-running extraction and detection jobs run as background tasks through the Redis queue. The engineer submits a batch, closes their browser, attends a meeting, and returns to find the results ready for download. No job is lost if the browser session ends. The system is designed for the actual working patterns of engineering professionals, not the idealised workflows of demos.

Chapter Eight: From Drawing to Deliverable - The Workflow in Action

The proof of any engineering tool is not its architecture - it is what the engineer experiences when they use it.

A XYRA Studio instrumentation workflow begins with an upload. The engineer drops one or more P&ID PDFs into the workspace. XYRA accepts vector PDFs with embedded text, rasterised PDFs created from scanned drawings, and mixed batches of both. There is no preprocessing required, no file renaming convention to follow, no project setup wizard.

The uploaded drawings appear as tabs in the workspace. The engineer can preview any drawing before running extraction, set an area code if their project uses location-based discipline areas, and optionally provide a project context - the legend or scope sheet from the drawing set - to help XYRA understand project-specific naming conventions.

The engineer submits the extraction job. The system queues it and begins processing in the background.

For each drawing, XYRA applies its pipeline in sequence. Vector text extraction or OCR, depending on the PDF type, produces a raw corpus of text found in the drawing. The instrument identification model filters this corpus, separating instrument tags from line numbers, equipment labels, title block text, and noise fragments. Each surviving tag is passed to the classification model, which applies ISA-5.1 decode logic and assigns an instrument type with a confidence score. The line mapper correlates each instrument with nearby pipe line numbers using geometric proximity and drawing context. The service enricher infers a description from the surrounding text and project context.

The result is a structured record for each instrument. Every record carries its raw extracted text, its cleaned tag number, its classified instrument type, its loop association, its connected line number, its service description, its confidence scores, and any QA flags - uncertainty markers that tell the reviewing engineer exactly why a record deserves attention.

When the batch completes, the engineer downloads a ZIP package. Inside: the Instrument Index in Excel, the IO List, the Verification Log, and the Line List. The Excel files are formatted for immediate use - not as data exports, but as engineering deliverables ready for review, sharing, and submission.

The piping MTO workflow follows a similar pattern. The engineer opens a drawing, identifies a valve type, and draws a selection box. XYRA captures the template, trims external whitespace, and stores it in the component library. The engineer repeats this for each distinct component type - gate valves, ball valves, check valves, spectacle blinds, strainers, specialty items. Then they run detection: XYRA scans every drawing in the batch, every page, every orientation, and returns a count with page-level evidence for every match.

In the detection review interface, the engineer can inspect every result, zoomed into the drawing at the detection location. False positives are removed with a click. The remaining results are confirmed

and exported as the MTO package.

What used to take weeks takes hours. What used to require a dedicated take-off engineer working in isolation now happens in a shared workspace where results are immediately available to every discipline.

Chapter Nine: The Centralized Engineering Database - The Bigger Vision

Everything described so far - InstruMap, Piping MTO, PrecisionPDF, the local LLM stack - is XYRA Studio as it exists today. It is already a significant advance over the state of the art in EPC engineering tooling.

But the founding vision of XYRA AI extends further. The extraction and detection capabilities are the beginning. The destination is a centralized engineering database: a structured, shared, revision-aware record system that captures every instrument, every loop, every pipe line, every piping component - across every drawing, every revision, every discipline - in a single queryable store.

The target technology is PostgreSQL. Not because it is exotic, but because it is the right tool: robust, open-source, multi-user, ACID-compliant, and capable of growing from a single-project deployment to an enterprise-scale engineering intelligence platform. PostgreSQL is free. It has no per-seat licensing. It runs on the same server as the rest of the XYRA stack.

The centralized database changes the nature of the tool. Instead of session-level extraction - where results exist as downloaded files and the system's memory ends when the job completes - every extraction run is stored as a versioned record. The same instrument tag across three drawing revisions becomes three records, linked, with the differences visible and auditable.

The piping engineer who needs the line list does not request an email. They query the database. The controls engineer building IO allocation tables does not copy from a spreadsheet. They pull from the same structured store that the instrumentation engineer used to build the Instrument Index. The project manager reviewing progress sees live counts, not static reports that are already out of date by the time they are read.

And underneath this shared record, the foundation for a new kind of engineering intelligence: retrieval-augmented AI. An engineer asking "list all pressure transmitters on lines larger than four inches classified as hazardous area" does not need to write a SQL query. They ask the question. XYRA retrieves the answer from the structured record, with drawing references, revision history, and confidence indicators.

This is not a speculative future. It is the logical extension of what XYRA Studio already does: transform engineering drawings into structured, queryable knowledge. The centralized database is the vessel that holds that knowledge, makes it available to every discipline simultaneously, and gives every AI query a factual foundation instead of a hallucination.

Chapter Ten: The Future of EPC Intelligence

The EPC industry is at an inflection point.

The generation of engineers who performed manual take-off as a matter of course is nearing retirement. The generation entering the industry has grown up with AI-assisted tools in every other domain of their lives. They do not accept that the only way to build an Instrument Index is to type it, row by row, from a PDF.

At the same time, the projects themselves are growing in complexity. Net-zero industrial transitions require new plants, retrofitted facilities, and expanded infrastructure at a pace and scale the industry has rarely encountered. The demand for EPC engineering capacity is increasing faster than the supply of experienced engineers.

The only resolution to this constraint is tools that make each engineer more productive - not tools that replace engineers, but tools that eliminate the clerical work that prevents engineers from doing engineering.

XYRA AI is positioned at exactly this intersection. It does not attempt to replace the instrumentation engineer's judgment. The engineer still reviews every extraction, removes false positives, resolves ambiguous classifications, and approves the final deliverable. What XYRA removes is the prior step: the hours of manual extraction that produced a first draft that was already partially wrong and had to be verified before any engineering judgment could be applied.

The roadmap ahead for XYRA AI includes capabilities that extend the platform in directions that are natural evolutions of the current foundation: per-project component libraries for MTO, material class mapping from project valve specifications, enhanced project legend processing, and the centralized database infrastructure described in the previous chapter. Longer-term, customer-specific model adapters - fine-tuned on a client's own verified extraction data - will produce extraction accuracy that improves project by project, drawing on the accumulated evidence of every job XYRA has processed for that client.

Custom AI Solutions - setting up EPC projects on Hexagon and AVEVA tools, building bespoke engineering applications, automating project-specific workflows, training models on client data, developing custom DLLs for Hexagon and AVEVA applications, protecting those extensions through DLL obfuscation, and delivering controlled engineering data services - extend XYRA's role beyond a product into a partner for EPC firms that want to build proprietary AI capability inside their own organisations.

The model is a monthly subscription: clean, predictable, with no per-seat licensing, no usage metering, and no data leaving the client's environment. A single server deployment serves an entire engineering team. The economics are straightforward: XYRA costs a fraction of the engineering time it saves on every project it touches.

Afterword: Why This Matters

This book has described a technical product and the engineering workflow problem it solves. But the reason XYRA AI exists - the deeper reason - is simpler than any of the technical arguments made in the preceding chapters.

Experienced engineers are scarce and expensive. The work they are trained to do - design safe systems, make sound engineering judgments, solve hard technical problems - is irreplaceable. No AI built today can tell you whether a process design is fundamentally sound, whether a safety shutdown logic is correctly specified, or whether a novel plant configuration will behave as intended under abnormal conditions.

What AI can do - what XYRA AI does - is handle the parts of the work that have nothing to do with engineering judgment. Copying tag numbers. Re-entering data into different systems. Counting valve instances on drawings. Assembling lists. Formatting outputs.

These tasks do not require engineering judgment. They require time, accuracy, and patience. Spending engineering expertise on them is not just inefficient - it is a misallocation of one of the most valuable and limited resources in the industry.

XYRA AI was built on a simple conviction: that engineering capacity should be spent on engineering. That the instrumentation engineer who spent four hours extracting tags from a P&ID should have spent those four hours reviewing them - verifying the safety logic, checking the control philosophy, catching the design error that extraction would never have found.

The Last Spreadsheet, the one that replaces the manual Instrument Index, has not yet been written. But the tool that will make it unnecessary is live, deployed, and processing drawings in EPC offices today.

That tool is XYRA Studio. The company behind it is XYRA AI.

And this is only the beginning.

XYRA AI - Private AI for EPC Engineering

www.xyra-ai.com

© 2026 XYRA AI. All rights reserved.

This document is for informational purposes. Contents are confidential.

XYRA AI - Private AI for EPC Engineering

www.xyra-ai.com